

Лямбды (замыкания) в Rust

Иванов Иван Иванович

Санкт-Петербургский государственный университет
Кафедра системного программирования

2025

- 1 Введение
- 2 Базовый синтаксис
- 3 Трейты Fn, FnMut, FnOnce
- 4 Захват переменных
- 5 Замыкания как аргументы и возвращаемые значения
- 6 Итераторы и замыкания
- 7 Практические паттерны
- 8 Заключение

Что такое замыкание?

- **Замыкание** (closure) — анонимная функция, которая может **захватывать переменные** из окружающего контекста
- В Rust замыкания — полноценные объекты, реализующие трейты `Fn`, `FnMut` или `FnOnce`
- Компилятор автоматически определяет тип захвата и нужный трейт

Синтаксис замыкания

|параметры| { тело } **или** |параметры| выражение

```
// Function -- explicit, named  
fn add_fn(x: i32, y: i32) -> i32 { x + y }
```

```
// Closure -- anonymous, captures environment  
let add = |x: i32, y: i32| -> i32 { x + y };
```

```
// Type inference -- even shorter  
let add_short = |x, y| x + y;
```

Замыкания vs функции в Rust

Свойство	Функция	Замыкание
Захват переменных из среды	Нет	Да
Вывод типов аргументов	Нет	Да
Хранение в переменной	Указатель	Анонимный тип
Передача как аргумент	fn ptr	impl Fn / dyn Fn
Размер известен в compile-time	Да	Да (impl) / Нет (dyn)

```
fn main() {  
    // No parameters  
    let greet = || println!("Hello, Rust!");  
    greet(); // Hello, Rust!  
  
    // With parameters and type inference  
    let square = |x| x * x;  
    println!("{}", square(5)); // 25  
  
    // With explicit types and block  
    let divide = |a: f64, b: f64| -> f64 {  
        if b == 0.0 { panic!("Division by zero"); }  
        a / b  
    };  
    println!("{:.2}", divide(10.0, 3.0)); // 3.33  
  
    // Capture variable from scope  
    let base = 10;  
    let add_base = |x| x + base; // base captured by reference  
    println!("{}", add_base(5)); // 15  
}
```

Треиты замыканий: Fn, FnMut, FnOnce

Каждое замыкание автоматически реализует один или несколько трейтов в зависимости от того, как оно использует захваченные переменные.

Треит	Когда реализуется	Вызов
FnOnce	Всегда (базовый трейт)	Один раз, потребляет захват
FnMut	Изменяет захваченные данные	Многократный, <code>&mut self</code>
Fn	Только читает захваченные	Многократный, <code>&self</code>

Иерархия трейтов

$\text{FnOnce} \supset \text{FnMut} \supset \text{Fn}$

Если замыкание реализует Fn, оно также реализует FnMut и FnOnce

FnOnce — однократный вызов

Замыкание **потребляет** захваченное значение — может быть вызвано только один раз, так как значение перемещается внутрь.

```
fn call_once<F: FnOnce()>(f: F) {
    f(); // ownership of f moved here
    // f(); -- ошибка: use of moved value
}

fn main() {
    let name = String::from("Rust");

    // name moved into closure
    let greet = || println!("Hello, {}!", name);

    call_once(greet);
    // greet(); -- ошибка: use of moved value `greet`
    // println!("{}", name); -- ошибка: name перемещена в замыкание
}
```

Типичное применение

Однократные операции: инициализация, порождение, владение ресурсом, потоки

FnMut — изменяющее замыкание

Замыкание **изменяет** захваченные данные. Переменная, хранящая такое замыкание, должна быть объявлена как `mut`.

```
fn apply_mut<F: FnMut()>(mut f: F) {  
    f();  
    f();  
    f();  
}  
  
fn main() {  
    let mut count = 0;  
  
    // count captured by mutable reference  
    let mut increment = || {  
        count += 1;  
        println!("count = {}", count);  
    };  
  
    increment(); // count = 1  
    increment(); // count = 2  
    increment(); // count = 3
```


Fn — иммутабельное замыкание

Замыкание только **читает** захваченные данные — наиболее гибкий трейт, допускает многократный вызов из нескольких мест.

```
fn apply_twice<F: Fn(i32) -> i32>(f: F, x: i32) -> i32 {
    f(f(x)) // f called twice
}

fn main() {
    let multiplier = 3;

    // multiplier captured by immutable reference
    let triple = |x| x * multiplier;

    println!("{}", apply_twice(triple, 2)); // triple(triple(2)) = 18

    // Fn closure can be called many times
    let values = vec![1, 2, 3, 4, 5];
    let doubled: Vec<i32> = values.iter().map(|&x| x * multiplier).collect();
    println!("{:?}", doubled); // [3, 6, 9, 12, 15]

    println!("multiplier = {}", multiplier); // variable still accessible!
}
```

Способы захвата: по ссылке и по значению

По умолчанию компилятор выбирает **минимально необходимый** способ захвата. Ключевое слово `move` принудительно перемещает все захваченные значения.

```
fn main() {  
    let text = String::from("hello");  
  
    // Default -- captured by reference  
    let print_ref = || println!("{}", text);  
    print_ref();  
    println!("text ещё доступна : {}", text); // OK  
  
    // move -- captured by value  
    let print_owned = move || println!("{}", text);  
    print_owned();  
    // println!("{}", text); -- ошибка: text перемещена в замыкание  
}
```

Когда нужен `move`?

- Замыкание передаётся в новый поток (`thread::spawn`)
- Замыкание возвращается из функции и переживает свой контекст

move-замыкания и потоки

```
use std::thread;

fn main() {
    let message = String::from("Hello from thread!");

    // Without move -- error: closure may outlive main
    // thread::spawn(|| println!("{}", message));

    // move -- message moved into thread
    let handle = thread::spawn(move || {
        println!("{}", message);
    });

    handle.join().unwrap();
    // println!("{}", message); -- ошибка: message перемещена
}
```

```
// Multiple vars -- all captured via move
fn make_adder(x: i32) -> impl Fn(i32) -> i32 {
    move |y| x + y // x moved into closure
}
```

Замыкания как аргументы функций

Статическая диспетчеризация (`impl Fn`) — тип мономорфизируется в compile-time:

```
fn apply<F: Fn(i32) -> i32>(f: F, x: i32) -> i32 { f(x) }

let double = |x| x * 2;
let square = |x| x * x;
println!("{}", apply(double, 5)); // 10
println!("{}", apply(square, 5)); // 25
```

Динамическая диспетчеризация (`dyn Fn`) — тип стирается, выбор в runtime:

```
fn apply_dyn(f: &dyn Fn(i32) -> i32, x: i32) -> i32 { f(x) }

let ops: Vec<Box<dyn Fn(i32) -> i32>> = vec![
    Box::new(|x| x * 2),
    Box::new(|x| x * x),
    Box::new(|x| x + 10),
];
for op in &ops { println!("{}", apply_dyn(op.as_ref(), 5)); }
// 10, 25, 15
```

Возврат замыкания из функции

Замыкание имеет анонимный тип, поэтому для возврата используется `impl Fn` (статически) или `Box<dyn Fn>` (динамически).

```
// impl Fn -- concrete type known at compile time
fn make_multiplier(factor: i32) -> impl Fn(i32) -> i32 {
    move |x| x * factor
}

// Box<dyn Fn> -- type erased, chosen at runtime
fn make_op(kind: &str) -> Box<dyn Fn(i32) -> i32> {
    match kind {
        "double" => Box::new(|x| x * 2),
        "square" => Box::new(|x| x * x),
        _        => Box::new(|x| x),
    }
}

fn main() {
    let triple = make_multiplier(3);
    println!("{}", triple(7));    // 21
    println!("{}", triple(10));   // 30
}
```

Итераторы и замыкания — идиоматичный Rust

Методы итераторов принимают замыкания и активно применяются для функциональной обработки коллекций.

```
fn main() {  
    let numbers = vec![1, 2, 3, 4, 5, 6, 7, 8, 9, 10];  
  
    // map -- преобразованиекаждогоэлемента  
    let squares: Vec<i32> = numbers.iter()  
        .map(|&x| x * x)  
        .collect();  
  
    // filter -- отборэлементовпопредикату  
    let evens: Vec<i32> = numbers.iter()  
        .filter(|&&x| x % 2 == 0)  
        .collect();  
  
    // fold -- свёрткааналог ( reduce)  
    let sum = numbers.iter()  
        .fold(0, |acc, &x| acc + x);  
  
    println!("{:?}", squares); // [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]  
    println!("{:?}", evens);   // [2, 4, 6, 8, 10]
```

Цепочки итераторов

Методы итераторов **ленивые**: промежуточные замыкания не выполняются до вызова терминального метода (`collect`, `for_each`, `sum...`).

```
fn main() {
    let result: Vec<String> = (1..=10)
        .filter(|x| x % 2 == 0)           // keep even numbers
        .map(|x| x * x)                   // square them
        .filter(|x| *x > 10)              // filter > 10
        .map(|x| format!("val={}", x))    // convert to string
        .collect();

    println!("{:?}", result);
    // ["val=16", "val=36", "val=64", "val=100"]

    // any / all -- проверка предиката
    let has_large = (1..=10).any(|x| x > 8);
    let all_pos    = (1..=10).all(|x| x > 0);
    println!("{}", has_large, all_pos); // true true

    // flat_map -- отображение с разворачиванием
    let words = vec!["hello world", "foo bar"];
    let chars: Vec<&str> = words.iter()
```

Паттерн: передача стратегии

Замыкания позволяют реализовывать паттерн **Стратегия** без создания трейтов и отдельных структур.

```
fn sort_by_strategy<T, F>(data: &mut Vec<T>, strategy: F)
where
    F: Fn(&T, &T) -> std::cmp::Ordering,
{
    data.sort_by(strategy);
}

#[derive(Debug)]
struct Person { name: String, age: u32 }

fn main() {
    let mut people = vec![
        Person { name: "Alice".into(), age: 30 },
        Person { name: "Bob".into(), age: 25 },
        Person { name: "Carol".into(), age: 35 },
    ];

    // Sort by age
    sort_by_strategy(&mut people, |a, b| a.age.cmp(&b.age));
}
```


Паттерн: мемоизация и ленивые вычисления

```
use std::collections::HashMap;

struct Memoize<F, A, B> {
    func: F,
    cache: HashMap<A, B>,
}

impl<F, A, B> Memoize<F, A, B>
where
    F: Fn(A) -> B,
    A: std::hash::Hash + Eq + Copy,
    B: Copy,
{
    fn new(func: F) -> Self {
        Memoize { func, cache: HashMap::new() }
    }
    fn call(&mut self, arg: A) -> B {
        if let Some(&result) = self.cache.get(&arg) {
            return result;
        }
        let result = (self.func)(arg);
        self.cache.insert(arg, result);
    }
}
```

Рассмотренные возможности замыканий в Rust:

- **Базовый синтаксис** — краткая запись, вывод типов
- **Треиты Fn / FnMut / FnOnce** — система типов гарантирует корректность захвата
- **Захват переменных** — по ссылке по умолчанию, `move` для передачи владения
- **impl Fn vs dyn Fn** — статическая и динамическая диспетчеризация
- **Итераторы** — ленивые цепочки `map` / `filter` / `fold`
- **Паттерны** — стратегия, мемоизация, фабрики функций

Ключевое отличие от других языков

Система владения Rust делает захват переменных **безопасным по памяти** на уровне типов — никакого GC

Когда использовать замыкания?

- Итераторы и функциональный стиль
- Колбэки и обработчики событий
- Паттерн Стратегия
- Многопоточность (`thread::spawn`)